Compilation On The GPU? A Feasibility Study

Robin F. Voetter
LIACS, Leiden University
Leiden, The Netherlands
robin@voetter.nl

Marcel Huijben
LIACS, Leiden University
Leiden, The Netherlands
marcelhuij@live.nl

Kristian F. D. Rietveld
LIACS, Leiden University
Leiden, The Netherlands
k.f.d.rietveld@liacs.leidenuniv.nl

ABSTRACT

The emergence of highly parallel architectures has led to a renewed interest in parallel compilation. In particular, the widespread availability of GPU architectures raises the question whether compilation on the GPU is feasible. In this paper, we describe the first design and implementation of a parallel compiler from a simple imperative programming language to RISC-V machine code, that is fully executed on a GPU. To accomplish this, all stages from parsing to machine code generation were redesigned to exploit fine-grained parallelism. Experimental evaluation of the implemented prototype demonstrates our proposed parallel techniques to be effective and implementation of compilation on the GPU to be feasible. Finally, we propose a number of avenues for future work and hope to revitalize research into parallel compilation conducted in the 1980s.

CCS CONCEPTS

• **Software and its engineering** → **Compilers**; • **Theory of computation** → *Parallel algorithms.*

KEYWORDS

Parallel Compilation, GPUs, Compiler Construction, Parsing, Code Generation

ACM Reference Format:
Robin F. Voetter, Marcel Huijben, and Kristian F. D. Rietveld. 2022. Compilation On The GPU? A Feasibility Study. In *19th ACM International Conference on Computing Frontiers (CF'22), May 17–19, 2022, Torino, Italy.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3528416.3530249

1 INTRODUCTION

In the 1980's and 1990's parallel compilation was an active area of research [7, 15, 16]. Despite these efforts, a fully parallel compiler has never been realized. Currently, parallelization of the compilation process is typically achieved at a coarse-grained level by processing different source files (translation units) at the same time, or by performing analysis on compilation units in parallel. With the stagnation of the increase in single-core performance [18] and the emergence of highly parallel architectures, compiler design warrants to be reconsidered. In fact, there is a renewed interest in parallel compilation by taking the parallelization to an even finer-grained level [3, 10, 19].

This work is licensed under a Creative Commons Attribution International 4.0 License.

CF'22, May 17–19, 2022, Torino, Italy
© 2022 Copyright held by the owner/author(s).
ACM ISBN 978-1-4503-9338-6/22/05.
https://doi.org/10.1145/3528416.3530249

We are in particular interested in the feasibility of performing the entire compilation process on Graphics Processing Units (GPUs), which are now widely available. This might pave the way to accelerate the compilation process by taking advantage of features introduced in hardware in the last years, such as SIMD units and GPUs, or to offload compilation to GPUs contrary to preparing executable GPU code on the CPU beforehand.

In this paper, we present a feasibility study on implementing and executing the full compilation chain on the GPU. More specifically, we describe the first design and implementation of a parallel compiler from a simple imperative programming language to RISC-V machine code. We have devised our own simple programming language due to the limitations of current parallel parsers and the scope of this study. This language serves as a demonstrator that implementation of a fully parallel compiler, by redesigning all compilation stages to exploit fine-grained parallelization, is feasible. To the best of our knowledge, our implementation is the first to perform all of these steps in parallel on single translation units. We extend and adapt prior work on parallel lexing and parsing [7, 15, 20] and describe our implementation of parallel semantic analysis and code generation, including register allocation.

Subsequently, we perform an experimental evaluation to see whether compilation on the GPU is advantageous from a performance perspective. Experiments conducted on a modern GPU architecture show a sub-linear scaling of the execution time against a linear increase in the size of the source file for the majority of the compiler stages. This shows our proposed parallel techniques to be effective. In fact, for large input files better scaling compared to traditional CPU-based compilers is already seen, showing the potential of GPU-based compilation. Based on these initial results, we identify a number of avenues for future work, in particular to improve the register allocation stage which is shown to be a bottleneck. By doing so, there is the potential to lower the bound on the input file size for GPU compilation to be profitable. Furthermore, due to the portable nature of our implementation, the study can be extended to many-core SIMD CPU architectures in the future.

Contributions. Our work makes the following contributions:

- We describe an implementation of a compiler running completely on the GPU, which is capable of taking a program written in a custom designed imperative programming language and compiling it into RISC-V instructions.
- We evaluate the effectiveness of the design and give an indication of the performance compared to traditional compilation with a number of experiments.
- Based on these results, we identify avenues for future work to revitalize research into parallel compilation techniques.
- We have released the compiler as open source, available at https://github.com/Snektron/pareas.

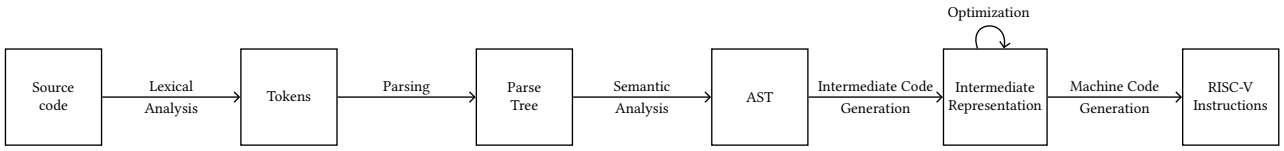


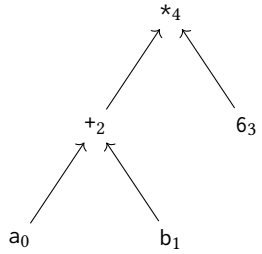
Figure 1: Schematic overview of the compiler passes that are considered in this paper.

This paper is organized as follows. Section 2 describes the design of our parallel compiler and Section 3 its implementation on modern GPUs. Section 4 presents an evaluation of the compiler's parallel performance. Section 5 describes future work. Section 6 discusses related work in parallel compilation. Section 7 concludes the paper.

2 DESIGN

In this paper, we consider the compilation passes from parsing of the source code of a program until the generation of machine code, summarized in Figure 1. To parallelize a compiler there are two approaches: run many, possibly different, passes in parallel, or parallelize each pass individually. Given the SIMD nature of GPU architectures, the latter is the more suitable choice, and we describe the individual parallelization of each pass. Another important property of GPU architecture is the reliance on streaming memory access and that codes involving frequent branching and irregular memory access (e.g., due to indirection arrays or pointer-linked data structures) are ill suited. In fact, traditional compilers are for the most part centered around the manipulation of pointer-linked trees using recursive functions that involve heavy branching. Therefore, in our design an inverted tree data structure is used instead of pointer-linked trees. The inverted tree is array-based and can be manipulated in a straightforward manner using parallel operations.

Figure 2 illustrates the differences between these. The inverted tree was recently proposed for the implementation of compiler transformations on the GPU by Hsu, et al. [10]. We use a variant of the inverted tree and store the parse tree in pre-order and Abstract Syntax Tree (AST) in post-order. Additionally, we use different algorithms to manipulate the inverted tree. Whereas pointer-linked trees are manipulated using recursive operations, we use iterative, loop-based array accesses which are easier to parallelize. The basic inverted tree data structure is augmented with additional fields and node types as the compilation progresses.



index	type	parent	depth
0	deref	2	2
1	deref	2	2
2	add	4	1
3	constant	4	1
4	mul	-1	0

Figure 2: Left: illustration of regular tree for expression `(a+b) * 6`, subscript indicates node index. Right: the corresponding inverted tree data structure in post-order.

The parallel operations are implementing using parallel array primitives. This allows for a reduced development time compared to writing the various GPU kernels by hand and results in a portable code base as efficient implementations of these primitives exist for different architectures. A pure functional programming language, in our case Futhark [6], is used to compose these primitives into programs. In such languages no expression may have side effects and this makes it easier to reason about optimizing high-level constructs into parallel primitives. Many of the primitives that are used in the implementation, such as parallel map, reduction, scan and prefix sum were first described by Hillis and Steele [9].

2.1 Lexical Analysis

The first stage performs *lexical analysis* according to a *lexical grammar* to partition the input into a sequence of *tokens*. The main problem of parallel lexical analysis presents itself when grammars with more complicated tokens are introduced, such as comments and quoted strings. In such a case processors that are parsing sections of the input in parallel may need to look back to the start of the input to determine the current state, which is detrimental to the performance. We address this by representing the grammar as a single deterministic finite automaton that is run once for each token in the input. Subsequently, we employ the algorithm proposed by Hillis & Steele [9] to evaluate this automaton on all characters of the input string in parallel. This can be implemented in terms of lookup tables generated ahead of the lexical analysis, see Section 3.1.

2.2 Parsing

In the *parsing* stage the syntactic structure of the input is validated according to the language's syntactic grammar and a *parse tree* is built. This is usually performed using a *push-down automaton*, which requires the current state of the stack to be known to parse a section of the input. This causes subsequent sections to be dependent on each other, refraining these from being parsed in parallel. We break this dependence chain by making it possible to determine the top of the stack from the *previous* few input symbols, relative to the current input position, instead of maintaining an actual stack. Sections of the input are now processed in parallel by deriving the initial contents of the stack from preceding input symbols, parsing each section using a regular predictive *LL* parser and recording the push and pop operations. The recorded operations (*stack configuration changes*) are concatenated to validate the input and to produce the final parse. From the final parse the parse tree is constructed.

The foundation of a parallel parsing algorithm that takes this approach, and that we have implemented, is described in [20]. This algorithm is capable of deterministically parsing languages of the newly defined $LLP(q, k)$ grammar class in $O(\log n)$ parallel time, where n is the number of tokens to parse. Within this definition,

```
fn fib[x: int]: int {
    if x < 2 {
        return x;
    } else {
        return fib[x - 1] + fib[x - 2];
    }
}

fn main[a: int, b: float]: void {
    while (a + 2) < int(-b * 3.0) {
        a = fib[a];
    }
}
```

Figure 3: Example program in our programming language.

q and k denote the number of lookbehind symbols and lookahead symbols respectively. For this study, we have opted to implement an $LLP(1, 1)$ parser, in which case the stack configuration changes corresponding to a pair of lookbehind and lookahead symbols can be stored in a pre-computed two-dimensional table. In this case, the grammar must allow constructs to be deduced from two consecutive input symbols. This puts restrictions on the grammar which in turn leads to a limitation in the languages that can be parsed. One could modify the grammar to conform to these restrictions, but the result is a language with inconvenient and unorthodox syntax. Alternatively, one can modify the grammar to accept a super set of the desired syntax and verify full correctness during the semantic analysis stage that follows the parsing stage.

For the purpose of this initial feasibility study, we have designed a custom imperative and procedural programming language with a static type system, bearing resemblance to existing programming languages. As such, the operations performed in our parallel compiler relate to existing compilers. A super set of our language can be parsed using a $LLP(1, 1)$ parser and full correctness is verified in during semantic analysis. An example program is shown in Figure 3. As can be seen, we have had to make a number of compromises such as always enforcing curly braces around compound statements, using square brackets for function applications, parsing `else` and `elif` parts of `if`-statements as separate statements and verifying the relative order of these later, and limiting the number of supported data types. More work is necessary to strike a balance between parsing programming languages with traditional syntax and the degree of parallelism that can be achieved.

2.3 Semantic Analysis

This stage has three major tasks: (1) validate and correct the parse tree for the fact that a super set of the desired grammar was accepted by the parser, (2) analyze the parse tree to determine whether it fits the semantic rules of the language, (3) transform the parse tree to an AST. All of these tasks are implemented as a series of passes. A pass processes the parse tree and may add, replace or remove nodes. Each pass is designed such that it performs small, node-local operations that can be executed for all nodes of the tree in parallel, or by parallelizing large, tree-wide operations in a clever way.

2.4 Intermediate Code Generation

At this point, the AST is transformed into machine code. For this study, we have selected the RISC-V architecture as target because of the fixed-width instructions and the regularity of the instruction encoding. Part of the generation of instructions is the selection of free registers. Because this is highly dependent on neighboring instructions, this would severely restrict the degree of parallelism that can be achieved. To counter this, we postpone register allocation and first compile to an intermediate representation which uses virtual registers and that is similar to a static single assignment representation of RISC-V. Section 3.5 explains the allocation of physical registers. We allow each register to be written once and read multiple times, but do not allow a register to be written in between writes. The output register number is computed from the instruction location, already satisfying the requirement that each register can only be written once. The actual generation of the instructions is parallelized by processing all nodes at the same depth in the tree simultaneously.

2.5 Optimization

Optimizations can be performed on the intermediate representation that do not require information about the physical registers. Optimizations modify the instruction buffer and parallelization is done at the level of instructions instead of nodes. Many of the feasible optimizations at this stage involve the removal of instructions (e.g., dead code removal or constant folding). However, repeatedly removing instructions from an array-based instruction buffer is an expensive operation, as remaining instructions need to be scattered to new locations and function locations and jump targets need to be updated. Therefore, we employ an instruction mask indicating whether an instruction in the program is enabled, which later stages can use to determine whether an instruction will be optimized away. The actual removal of instructions and correction of data structures is delayed until the final step of the machine code generation.

2.6 Machine Code Generation

At this stage, the intermediate representation is transformed into the final RISC-V machine code. First, *register allocation* will map the infinite set of virtual registers onto the finite set of physical registers. If insufficient registers are available, registers will be flagged for spilling and the necessary stack space will be allocated. In traditional compilers a graph coloring algorithm is typically used to perform register allocation. While the graph coloring algorithm itself can be run efficiently on a parallel architecture like a GPU [5], the fact that each iteration may insert spill instructions would impose a significant overhead due to the array reallocation that is required. Therefore, we opt to implement a greedy algorithm to allocate registers. Although this does not result in mappings as optimal as those produced by a graph coloring approach, a reasonable mapping is still produced for most applications given the large amount of registers present on the RISC-V target architecture.

Additionally, the actual insertion of spill instructions is postponed until register allocation has completed. By doing so, all spill instructions can be inserted in one go *and* this can be combined with the removal of instructions according to the instruction mask. This results in the final instruction buffer. After post-processing the jump instructions and computing the final function offset table, the compilation process has finished and the results can be transferred from GPU to CPU memory.

3 IMPLEMENTATION

We have implemented a prototype of the described parallel compiler in a mix of C++ and Futhark [6] code. The former is used for the pre-processing tools to generate the pre-computed tables required for the lexical analyzer and parser, and for the compiler's driver code. The driver code loads these tables onto the GPU at runtime, along with the source code to compile in verbatim, after which it launches the compilation passes. All of the compilation passes are written in the Futhark functional programming language.

3.1 Lexical Analyzer & Parser

The pre-processing for the lexical analyzer and parser is combined into a single tool, which generates the appropriate data structures during the build process of the compiler. For the lexical analyzer, a single deterministic finite automaton that is run once for the entire input is constructed from a lexical grammar provided as text file. It is encoded into three (table) data structures to support simulation of the automaton using the algorithm of Hillis & Steele. To reduce the memory space required, we compute all reachable compositions of unary transition functions ahead of time and assign an (integer) identifier to each one of them.

To generate the parser, a syntactical grammar must be supplied in the form of a text file containing a number of productions. This input is processed into a total of three data structures. The generation algorithm that is employed is a straightforward implementation of the method discussed in [20]. As our compiler is designed around a $LLP(1, 1)$ grammar, we can pre-compute the stack configuration changes for every possible pair of token types and store this in a two-dimensional lookup table.

At runtime, the lexical analyzer automaton is evaluated using the algorithm of Hillis & Steele, implemented using parallel map for table lookups and a parallel prefix scan to deduce all token types that have been emitted during the evaluation. This sequence of token types is the final token stream, from which the parser will build the parse tree. The parallel parsing algorithm that we consider first maps each subsequent pair of symbols to stack configuration changes using the lookup table and a parallel string extraction algorithm. Secondly, the validity of the input is determined by verifying that the concatenated stack configuration changes are balanced using a parallel bracket matching algorithm [1]. Finally, the final parse can be computed in a similar fashion, but using a different lookup table and superstring. This is converted to a parse tree using a variant of depth-first traversal implemented using parallel primitives.

3.2 Semantic Analysis

This stage is implemented as a series of passes that analyze and manipulate the parse tree. This consists of passes such as extracting lexemes to literals, variable resolution, function resolution and type analysis. The implementation of these passes is facilitated by a set of common operations on the inverted tree data structure. For example, a typical operation is to find the root of certain subtrees. This operation can be implemented in parallel by applying the algorithm by Hillis & Steele [9] to find the end of a linked list on each of the nodes simultaneously. When the tree is converted to an AST, the inverted tree is converted from pre-order to post-order and a set of arrays is created where each array stores a particular node property (e.g., data type, parent index, depth of the node in the AST, literals). So, all information regarding a node is now easily reachable without tree walks, which simplifies the implementation of the subsequent code generation stages.

3.3 Intermediate Code Generation

First, the number of instructions to be generated is computed[1] and an instruction buffer is allocated. Second, a mapping is computed from nodes in the tree to instruction locations in the buffer. Both rely on a parallel map to map each node type to the number of instructions required to encode it. Small adjustments are made for special cases (e.g., branch nodes) and a function offset table, containing start and size of each function, is computed. To generate the actual instructions, child nodes must be processed before parent nodes. As such, a bottom-up tree walk is performed which will process all nodes at the same depth in parallel. By assuming each node will yield at most four instructions, we spawn four threads for each AST node. Each of these threads will determine instruction opcode, registers and branch target according to the node type.

3.4 Optimization

Simple optimizations can be implemented that amend instructions in the intermediate representation or mark instructions for removal in the instruction mask. As an example, consider dead expression removal, which removes any expression of which the result is not used. To do so, we repeatedly update a table that maps virtual register numbers to a boolean indicating whether that register is used, using parallel map and scatter operations. For instructions of which the destination register is not used, the booleans for the source registers are set to false. Because of the correspondence between virtual register numbers and instruction location, we can directly update the instruction mask from this table to indicate the instructions that can be optimized away. Instructions that induce side-effects are never removed.

3.5 Machine Code Generation

First, the physical register numbers to be encoded into the instructions must be determined. This register allocation starts with a lifetime analysis to determine which registers are in use at which point of the program. This analysis is performed in parallel for each function (and cannot be parallelized at the level of instructions). Each instruction of a function is visited sequentially and a lifetime mask, which contains a bit indicating availability of every physical register, is updated according to the operands. Physical registers are greedily assigned by finding a free register in the lifetime mask. If no free register is available, a register will be selected from a statically defined set of suitable scratch registers and marked as a spilled register. The assignment of virtual to physical registers is stored in the virtual register table together with a swap bit that identifies if that register needs to be spilled at any point.

After lifetime analysis has completed, stack space is allocated by computing a stack offset table that maps virtual registers to integer stack offsets. This computation is performed for all functions in parallel using the swap bits from the virtual register table.

[1] Note that a single node in the AST may result in more than one instruction.

Table 1: Properties of the generated input files.

File#	Size	Lines	Functions	AST Nodes
1	5.1 KB	121	5	1 211
2	11 KB	209	9	3 211
3	99 KB	1 992	80	26 946
4	500 KB	10 328	412	134 564
5	1.1 MB	21 939	895	289 082
6	9.8 MB	204 649	8387	2 675 153
7	47 MB	971 014	40112	12 872 847

Additionally, based on the lifetime mask a table is created that contains the number of instructions to be generated for each intermediate instruction including spill instructions. Subsequently, instructions are removed according to the instruction mask and spill instructions are inserted. These operations are combined for the sake of performance. New instruction locations are computed and all generated instructions are copied to the final locations in the instruction buffer using a parallel scatter. Afterwards, any required spill instructions are inserted obtaining the stack offsets from the stack offset table. With all instructions in place, the virtual register numbers in each instruction are resolved to physical registers using the virtual register table. As the final step, the function offset table is recomputed to account for the removed and inserted instructions, and the target addresses for all jump instructions are corrected. The separate buffers containing per-instruction information are merged to construct the final instruction buffer.

4 EVALUATION

For the purpose of this feasibility study, we are interested in the effectiveness of the parallel algorithms and an indication of the performance of the parallel compiler compared to traditional CPU-based compilation. To investigate the effectiveness, we have performed a number of experiments to determine the scaling characteristics under increasing input sizes. Given that a custom programming language is used in this study, there is no source code available that represents real-world use cases and translation of real-world source code to this language is complicated. Therefore, we have randomly generated seven source code files that we believe will yield results indicative of the performance on real-world source code. The properties of these files are shown in Table 1, all files are syntactically and semantically correct.

The experiments were conducted on an NVIDIA GeForce RTX3090 GPU, hosted in a machine equipped with dual Intel Xeon Silver 4214R processors. The software configuration consists of CentOS 7, Linux kernel 5.4.126, Futhark 0.20, CUDA 11.2 and gcc 10.2.0 in release mode (`-O3`). All runtime measurements are obtained using the C++ `chrono::high_resolution_clock`. Care is taken that GPU and CPU work is synchronized before a timestamp is collected. To reduce noise, we repeat every individual experiment 30 times.

Figure 4a depicts a breakdown of the runtime from parsing up to the creation of the AST. Almost all stages show similar characteristics. Initially, the scaling appears flat, as a relatively large amount of time is spent launching the kernels and combining the results of the individual shader processors, making the actual processing time almost negligible compared to the overhead. From the third to the sixth input file we see a sub-linear scaling, for example for Semantic Analysis we observe a factor 6.4 increase in runtime for a 100x increase in input size. From the sixth input file onwards this nears linear scaling, with a 4.7x increase in runtime for a 4.8x increase in input size.

The results for the code generation stages are shown in Figure 4b. Recall that the register allocation and instruction removal stages were merged. The majority of the stages show a scaling similar to the stages in Figure 4a, but a notable exception is the register allocation stage. As discussed in Section 3.5, lifetime analysis must be performed in parallel for each function, resulting in less parallelism being available to exploit, leading to a significant overhead compared to the other stages.

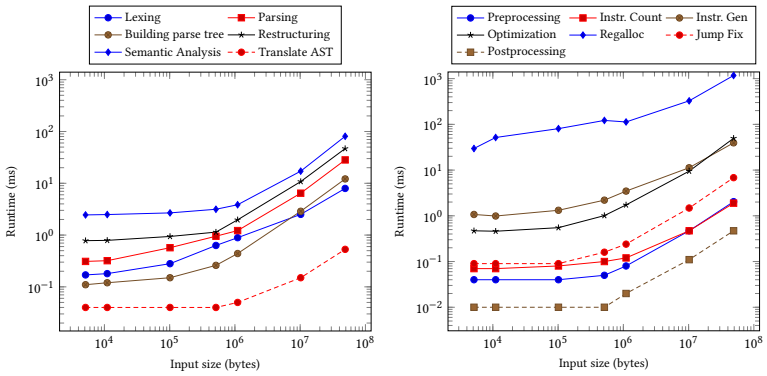
In Figure 5 the total runtime of the compiler is shown, excluding data transfer to/from the GPU. The total runtime is dominated by the register allocation stage. Still, a sub-linear scaling is seen, for instance from the fifth to seventh input file we observe an increase in runtime of 11.3x against a 44.5x increase in input size. This signifies that the parallelization of the different stages is effective.

A detailed performance comparison to other approaches is hard at this moment. On the one hand, given that we have presented the first compiler fully hosted on the GPU, there is no GPU baseline to compare to. A comparison to our compiler generated for the CPU would not yield a fair comparison as Futhark only generates multi-threaded CPU code, without using SIMD, causing not all available resources to be used. Moreover, all stages were designed with the high memory bandwidth available on GPUs in mind. On the other hand, a direct comparison to traditional CPU-based compilers would not be fair, as our compiler does not fully handle a programming language like C and does not implement all optimizations and instruction selection techniques. For a factual comparison the feature set of the GPU compiler must be up to par and also the quality of the produced code must be included in the comparison.

Still, we would like to obtain an initial performance comparison to see whether this development is worthwhile. To do so, we have translated the custom source files from the above experiments to C source files and measured the runtime of invoking `cc -w -c` on these files. The experiments were performed on an Intel Core i7-8700, using gcc 10.2.0 and Clang 8, and were repeated 10 times. The results are included as dashed lines in Figure 5. The C files are between 19.1% to 21.6% larger compared to the original files. As can be seen, the CPU-based compilers perform better for small input sizes, because of the overhead encountered by the GPU implementation. Once the input sizes get larger, the GPU implementation clearly shows better scaling compared to the CPU compilers. This shows that compilation on the GPU can be feasible with reasonable performance and that there is the potential to GPU-accelerate the compilation of large input files. Note that there is the potential to lower the bound on the input file size for GPU compilation to be profitable when register allocation is further optimized.

5 DISCUSSION AND FUTURE WORK

There are many avenues for further work. Most importantly, the identified performance bottlenecks need to be addressed to further increase the potential of GPU compilation. The large overhead of register allocation can possibly be alleviated by a redesign to operate on the level of expressions instead of functions. For the



(a) Parsing to semantic analysis. **(b) Code generation.**

Figure 4: Scaling characteristics of the compiler stages for increasing input size.

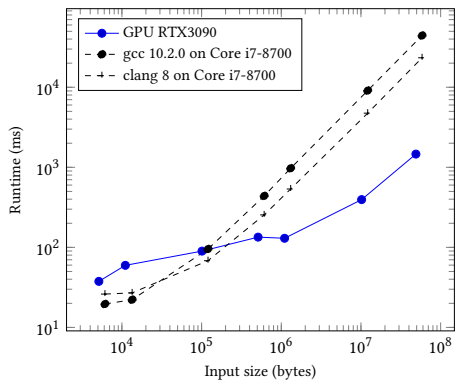


Figure 5: Total runtime of the GPU compiler and an indication of the runtime of traditional CPU compilers on similarly sized inputs.

code generation algorithms, only nodes at the same depth may be processed in parallel, limiting the degree of parallelism. By splitting the assignment of virtual registers out of the instruction generation step, it will become possible to execute nodes at different levels of the tree. These improvements will lead to a lower bound on the input size for GPU compilation to be profitable.

Besides performance improvements, work is needed on feature completeness. More research into parallel parsing techniques is required to shrink the number of restrictions posed on input grammars, to efficiently support the syntax of traditional languages. Also more complicated type systems should be supported. With regard to code generation, work is to be done on basic block detection, implementation of different optimizations and an evaluation of the quality of the produced code. Finally, we would like to evaluate our approach on many-core SIMD CPUs to determine whether this is worthwhile and what changes would be required for instance due to the limited memory bandwidth that is available.

6 RELATED WORK

Parallel compilation received considerable interest in the 1980's and 1990's, due to the development of parallel architectures such as the Connection Machine [8]. A good overview of the state-of-the-art in parallel compilation at that time, with a focus on parallel lexical and syntactical analysis, is given in [16]. In [11] an implementation of a parallel assembler is described, but this paper does not consider the transformation of an AST into assembly code. To the best of our knowledge, a fully parallel compiler, as described in our paper, was not realized at that time.

During the same time, parallel lexical analysis and parsing was investigated [7, 9, 15], where the focus was on the application of parallel prefix sum to simulate deterministic finite state machines. More recent work considered an implementation of SIMD hardware [13], CPU multicores [2] and methods to reduce the memory overhead [14]. A deterministic parallel parsing algorithm and the $LLP(q, k)$ grammar class was proposed by Vagner & Melcihar [20], which we have implemented as part of our compiler.

More recently, parallel compilation received renewed interest. The Parallel GCC project [3] modifies the GNU C Compiler to run function-local analysis and optimization stages on multiple CPU cores. Although a reasonable speedup is achieved, this only concerns a small part of the overall compilation process and the remainder is still single-threaded. CuLi is capable of executing a runtime environment for the Lisp programming language on GPUs by recursively evaluating the AST [17]. Contrary to our work, stages other than evaluation, such as parsing, are performed on the CPU.

Hsu describes the implementation of an APL to C++ compiler, which is completely hosted on the GPU, focusing on tree transformations performed by a compiler after parsing and before code generation [10]. This can be compared to the semantic analysis stage of our work. The inverted tree data structure follows a similar approach, but in our case a post-order storage is used and the tree traversal routines are partly based on [9]. Furthermore, contrary to Hsu, we present a full compiler hosted on the GPU and we describe solutions for the challenges of parallel parsing and parallel code generation, including register allocation.

Our prototype was developed in Futhark and differs from prior work on writing compilers in functional programming languages [4, 12]. Earlier work involves the use of language constructs, such as conditional branches, and data structures, such as lists into which elements are inserted repeatedly, that perform poorly on GPU architectures. Instead, we make extensive use of array-based data structures and parallel primitives.

7 CONCLUSIONS

We have described the first design and implementation of a parallel compiler that is fully executed on a GPU, demonstrating that it is feasible to implement all stages in a fine-grained parallel manner. The experimental evaluation showed the parallelization of all stages to be effective, although the total runtime is dominated by the register allocation stage. Compilation on the GPU can be done with reasonable performance, and in fact for large input files better scaling compared to traditional CPU-based compilers is already seen. We argue that compilation on the GPU is feasible and potentially worthwhile, and have proposed a number of avenues for future work hoping to further revitalize research into parallel compilation.

REFERENCES

[1] Ilan Bar-On and Uzi Vishkin. 1985. Optimal parallel generation of a computation tree form. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 7, 2 (1985), 348–357.

[2] Alessandro Barenghi, Stefano Crespi Reghizzi, Dino Mandrioli, Federica Panella, and Matteo Pradella. 2015. Parallel parsing made practical. *Science of Computer Programming* 112 (2015), 195–226.

[3] GNU Contributors. 2019. ParallelGcc. https://gcc.gnu.org/wiki/ParallelGcc

[4] Abdulaziz Ghuloum. 2006. An incremental approach to compiler construction. In *Proceedings of the 2006 Scheme and Functional Programming Workshop, Portland, OR. Citeseer*. Citeseer.

[5] Andre Vincent Pascal Grosset, Peihong Zhu, Shusen Liu, Suresh Venkatasubramanian, and Mary Hall. 2011. Evaluating graph coloring on GPUs. In *Proceedings of the 16th ACM symposium on Principles and practice of parallel programming*. 297–298.

[6] Troels Henriksen, Niels GW Serup, Martin Elsman, Fritz Henglein, and Cosmin E Oancea. 2017. Futhark: purely functional GPU-programming with nested parallelism and in-place array updates. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 556–571.

[7] Jonathan MD Hill. 1992. Parallel lexical analysis and parsing on the AMT distributed array processor. *Parallel computing* 18, 6 (1992), 699–714.

[8] W Daniel Hillis. 1989. *The connection machine*. MIT press.

[9] W Daniel Hillis and Guy L Steele Jr. 1986. Data parallel algorithms. *Commun. ACM* 29, 12 (1986), 1170–1183.

[10] Aaron Wen-yao Hsu. 2019. *A data parallel compiler hosted on the gpu*. Ph.D. Dissertation. Indiana University.

[11] Howard P. Katseff. 1988. Using Data Partitioning to Implement a Parallel Assembler. *SIGPLAN Not.* 23, 9 (Jan. 1988), 66–76. https://doi.org/10.1145/62116.62123

[12] Andrew W. Keep and R. Kent Dybvig. 2013. A Nanopass Framework for Commercial Compiler Development. *SIGPLAN Not.* 48, 9 (Sept. 2013), 343–350. https://doi.org/10.1145/2544174.2500618

[13] Todd Mytkowicz, Madanlal Musuvathi, and Wolfram Schulte. 2014. Data-parallel finite-state machines. In *Proceedings of the 19th international conference on Architectural support for programming languages and operating systems*. 529–542.

[14] Ryoma Sinya, Kiminori Matsuzaki, and Masataka Sassa. 2013. Simultaneous finite automata: An efficient data-parallel model for regular expression matching. In *2013 42nd International Conference on Parallel Processing*. IEEE, 220–229.

[15] David B Skillicorn and David T Barnard. 1989. Parallel parsing on the connection machine. *Inform. Process. Lett.* 31, 3 (1989), 111–117.

[16] David B. Skillicorn and David T. Barnard. 1993. Compiling in parallel. *J. Parallel and Distrib. Comput.* 17, 4 (1993), 337–352.

[17] Tim Süß, Nils Döring, André Brinkmann, and Lars Nagel. 2018. And now for something completely different: running Lisp on GPUs. In *2018 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 434–444.

[18] Herb Sutter and James Larus. 2005. Software and the Concurrency Revolution: Leveraging the full power of multicore processors demands new tools and new thinking from the software industry. *Queue* 3, 7 (2005), 54–62.

[19] Rui Ueyama. 2021. mold: A Modern Linker. https://github.com/rui314/mold/blob/main/README.md

[20] Ladislav Vagner and Bořivoj Melichar. 2007. Parallel LL parsing. *Acta informatica* 44, 1 (2007), 1–21.